On the Dynamics of Portfolio Optimization

Naseebullah Andar Samantha Agisim Patrick Ledoit

April 23, 2026

Why Dynamic Portfolio Optimization?

- Portfolio choice is a classical problem in economics: how to allocate capital across risky assets to improve return while controlling risk.
- In practice, optimization is fragile because:
  - expected returns are noisy,
  - covariance estimates are unstable,
- Simple rules such as equal weighting, under efficient market hypothesis, are therefore hard to beat out of sample.
- **Central Question:**

Can we develop an optimization framework that materially outperform equal weighting?

Model Setup

- At each date t, choose a portfolio weight vector w_t over the active universe.
- Inputs are estimated from a rolling historical window:
  - expected returns: $\hat{\mu}_t$
  - covariance matrix: $\hat{\Sigma}_t$
- The portfolio solves

$$\max_{w_t} \quad \hat{\mu}_t^\top w_t - \gamma\, w_t^\top \hat{\Sigma}_t w_t - \kappa \left\| w_t - w_{t-1} \right\|_1$$

- subject to

$$w_t \geq 0, \qquad \mathbf{1}^\top w_t = 1$$

- Interpretation:
  - reward expected return
  - penalize risk
  - penalize turnover

Data and Dynamic Universe

- Two equity return panels:
  - monthly data: 300 dates, 971 securities
  - daily data: 6289 dates, 978 securities
- The investable universe is *time-varying*.
- A stock enters the active universe only if it has enough prior history:
  - monthly model: 60 prior monthly returns
  - daily model: rescaled minimum-history requirement

- Additional screens:
  - common shares on major U.S. exchanges
  - optional low-price filter
  - at most 350 stocks per date

Baseline Results

- Relative to the equal-weight benchmark:
  - mean monthly return does *not* improve
- Interpretation:

The baseline optimizer behaves largely like a smoothed version of equal weighting rather than a strongly differentiated allocation rule.

Baseline Results

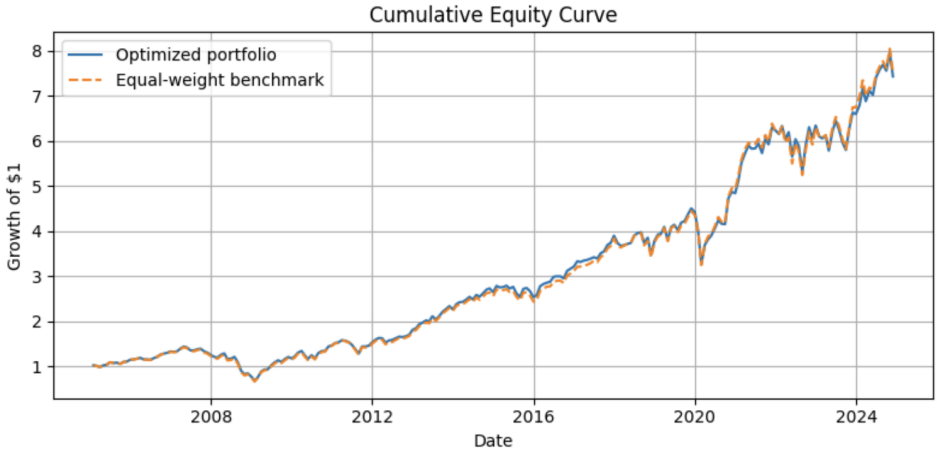


Figure 1:

Why Was the Baseline So Weak?

- A natural concern was that the optimizer might be mechanically stuck near equal weight.
- To test this, we ran a two-asset diagnostic:
  - one asset with relatively high estimated return
  - one asset with relatively low estimated return
- Under the baseline settings, the optimizer stayed close to 50/50.
- Under lighter regularization and more aggressive solver settings, it tilted almost entirely toward the better asset.

Recalibration Changes the Picture

- After diagnostic checks, we relaxed the economic penalties and gave the solver enough iterations to move meaningfully.
- A more aggressive low-penalty specification performed best:
  - mean monthly return rose to 0.0167
  - annualized Sharpe rose to 0.8204
  - cumulative return rose to 14.16

Recalibration Changes the Picture

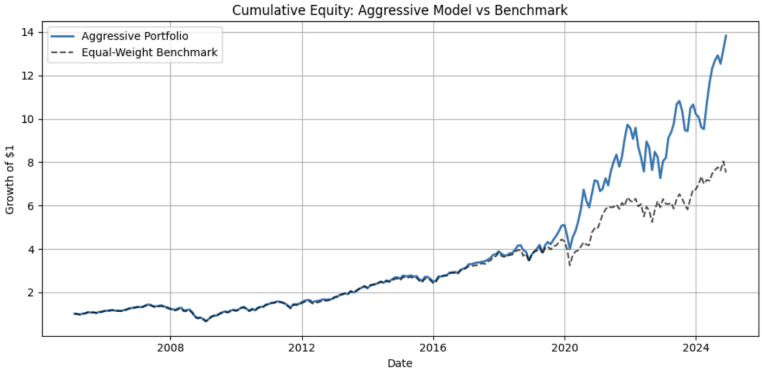
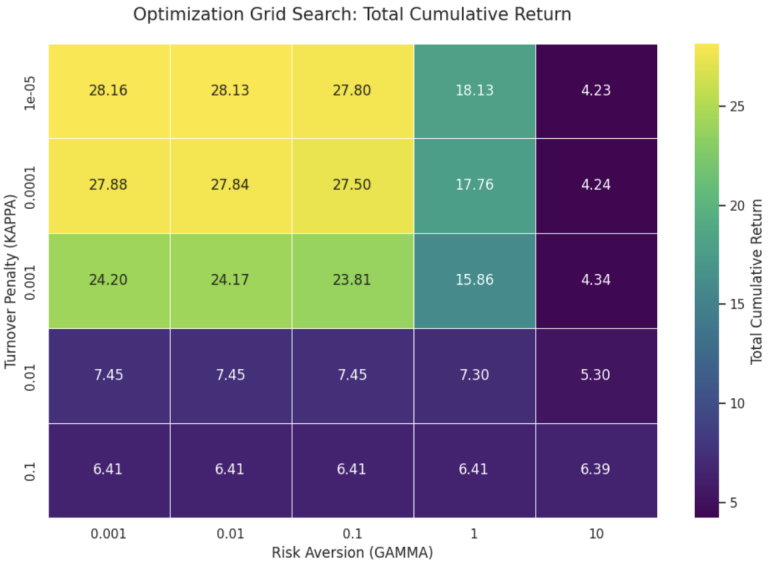


Figure 2:

What Regularization Works Best?

- We ran a 5×5 grid search over:
  - risk aversion γ
  - turnover penalty κ
- Main pattern:
  - the strongest results occur when both γ and κ are small
  - heavy regularization pushes the portfolio back toward equal-weight-like behavior

What Regularization Works Best?



What Regularization Works Best?

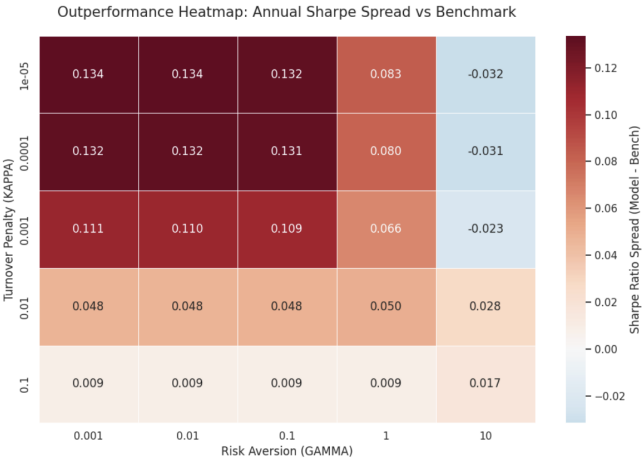


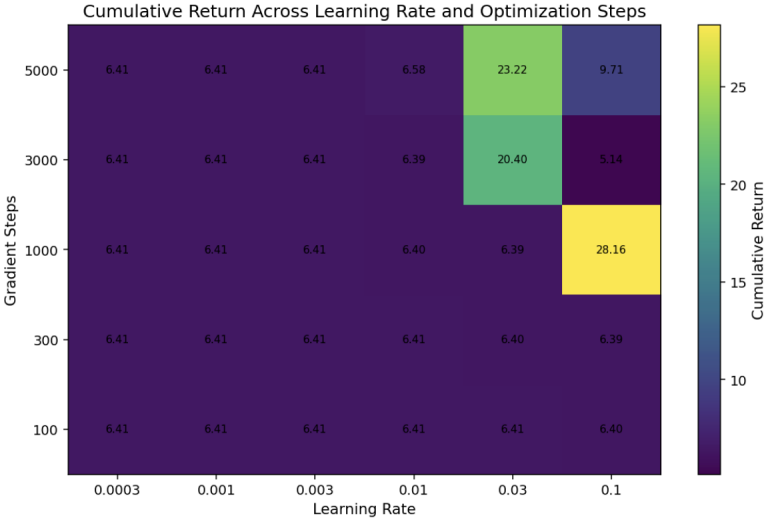
Figure 5:

Solver Tuning Matters

- Even with the same economic objective, performance changed sharply with:
  - learning rate
  - number of gradient steps
- Key finding:
  - very small learning rates left the optimizer near its initialization
  - very aggressive settings could overshoot and hurt performance
- Interpretation:

Numerical hyperparameters do affect the realized portfolio rule.

Solver Tuning Matters



How Much History Should We Use?

- We searched over rolling estimation windows:

$$L \in \{6, 12, 18, 24, 36, 48, 60\} \text{ months}$$

- Performance was *hump-shaped*:
  - very short windows were too noisy
  - very long windows were too inertial
- Interpretation:

The strongest results came from a middle ground between responsiveness and statistical stability.

How Much History Should We Use?

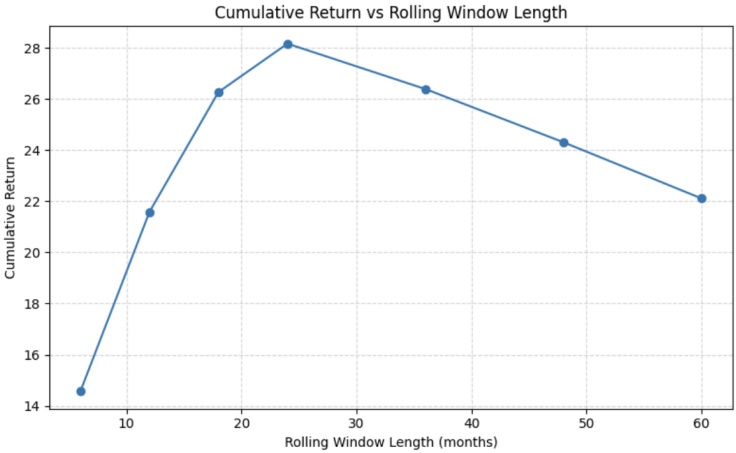


Figure 7:

Benchmark Comparison

- The equal-weight portfolio is the main *internal* benchmark:
  - same date-specific universe
  - same assets
  - same evaluation periods
- This isolates the value added by the optimization rule itself.
- We also compare the strategy to standard *external* market benchmarks:
  - S&P 500
  - Nasdaq 100
  - Russell 2000
- **Takeaway:**

The strongest optimized specifications do not just improve on equal weight; they also appear competitive with broad passive market portfolios.

Benchmark Comparison

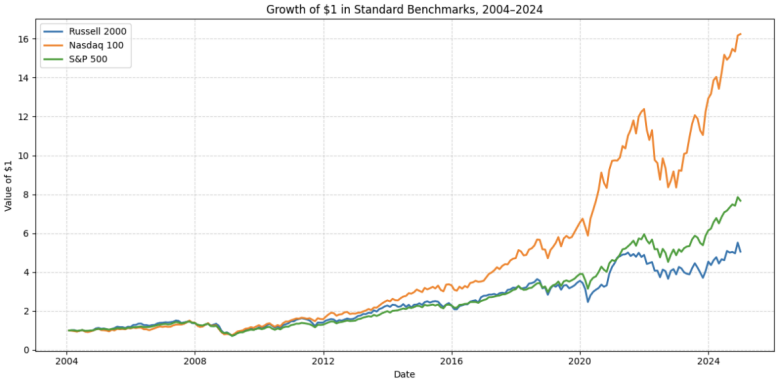


Figure 8:

Does the Framework Survive on Daily Data?

- We extended the model from monthly to daily returns.
- We rescaled the daily specification:

$$\text{ROLLING_WINDOW} = 504, \qquad \text{MIN_HISTORY} = 1260$$

- **Takeaway:**

The main mechanism survives at higher frequency.

Does the Framework Survive on Daily Data?

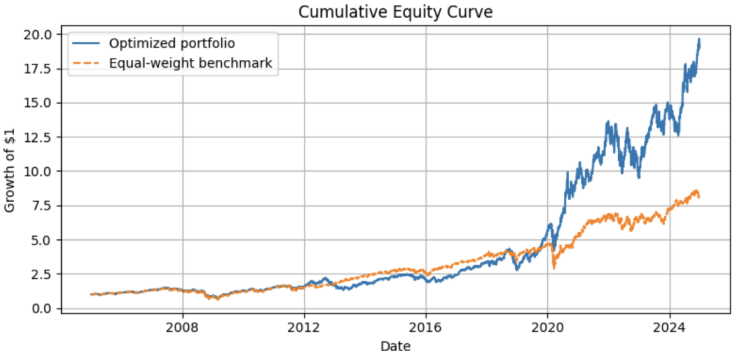


Figure 9:

Does the Framework Survive on Daily Data?

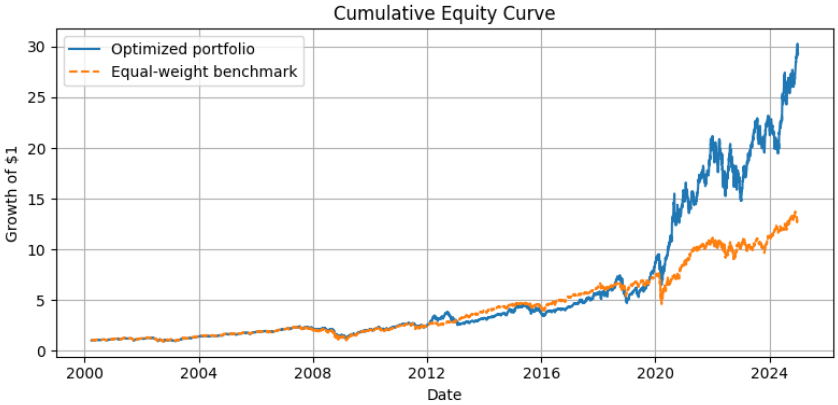
- We extended the model from monthly to daily returns.
- We didn't rescale the daily specification:

$$\text{ROLLING_WINDOW} = 36, \qquad \text{MIN_HISTORY} = 60$$

- **Takeaway:**

The main mechanism works even better with the same rules as the monthly optimization rules.

Does the Framework Survive on Daily Data?



Main Takeaways

- Portfolio optimization can outperform equal weighting, but only after careful calibration.
- The strongest monthly results came from:
  - light regularization
  - intermediate rolling windows
  - well-tuned solver settings
- The framework remained effective on daily data.
- **Overall lesson:**

The effective portfolio rule is determined jointly by the objective, the regularization, the estimation design, and the numerical solver.

Part II: Extending the Optimizer to Polymarket Data

- The second part of the project asks whether the same optimization logic can be extended beyond equities to prediction-market data.
- This is a different environment:
  - markets are shorter-lived,
  - liquidity is uneven,
  - prices can move discontinuously as information arrives.
- So the question is no longer just whether the optimizer beats equal weight.
- **New Central Question:**

Can a cost-aware, chronology-safe optimizer produce a useful trading signal on Polymarket data?

What changed in this version

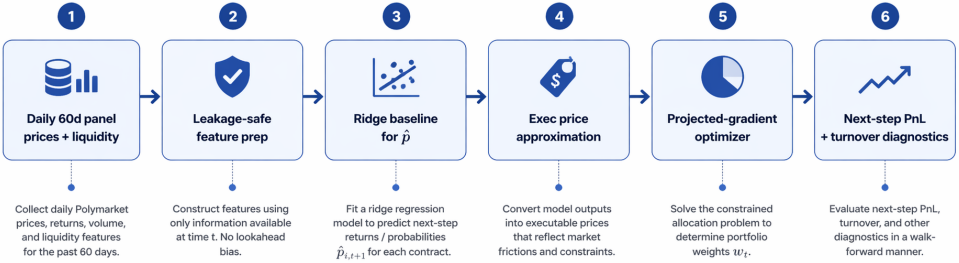
- This portion is refocused around the **60-day optimization problem**, not the broader panel experiment sweep.
- The narrative is rebuilt from two source docs:
  - `cursor_poly_market_exploration_using_da.md`
  - `polymarket_mispricing_pipeline.md`
- The goal is to make the workflow easy to follow end to end: **data** $\rightarrow$ **model** $\rightarrow$ **executable price** $\rightarrow$ **constrained optimizer** $\rightarrow$ **walk-forward diagnostics**.

Central question

Given the 60-day Polymarket panel, can a leakage-safe baseline model produce estimated probabilities that are strong enough to drive a constrained long-only mispricing portfolio?

Method in one line

End-to-end workflow: from daily Polymarket data to portfolio decisions and realized performance



Optimization problem

$$\max_{w} \sum_{i} w_i(\hat{p}_i - c_i^{\text{exec}}) - \gamma \sum_{i} w_i^2 \hat{p}_i(1 - \hat{p}_i) - \kappa \| w_t - w_{t-1} \|_1$$

Decision variables

- $w_i \geq 0$ for each contract
- cash is allowed, so $\sum_i w_i \leq 1$
- per-contract caps prevent concentration

Penalty terms

- γ controls variance-like risk via $\hat{p}_i(1 - \hat{p}_i)$
- κ penalizes turnover relative to the previous portfolio

Portfolio constraints

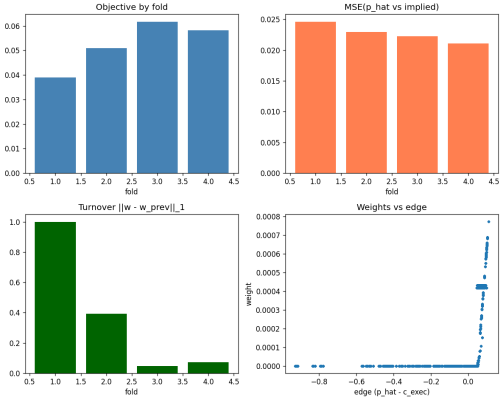
- event cap if contracts share an event
- liquidity load constraint
- projection step enforces feasibility after each gradient update

Implementation choice

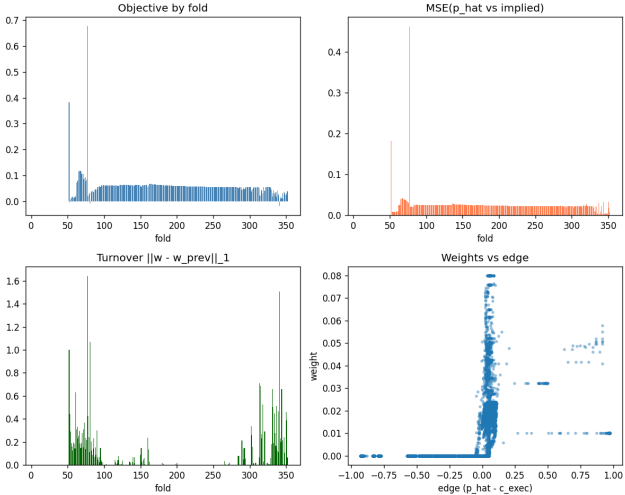
The repo uses projected subgradient ascent with a fast cyclic projection routine. That choice is simple, reproducible, and matches the design brief in the markdown docs.

Leakage-safe 60-day walk-forward design

- The 60-day multip run re-optimizes **day by day**.
- For each decision date, training uses only earlier rows.
- Test-time realized pnl is computed with **next-step implied price**, not the same-day quote.
- The summary file reports separate train, validation, and test aggregates.



Headline comparison: static 5-fold vs daily 60d re-optimization



Key numbers

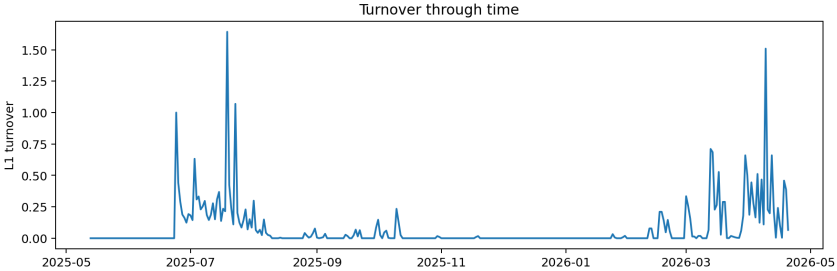
- Static 5-fold objective:
- Daily reopt objective:
- Static turnover:
- Daily reopt turnover:

Takeaways

The daily 60d re-optimization slightly lowers mean objective versus the static 5-fold summary, but it dramatically lowers turnover and is the more faithful operational test.

Turnover behavior and optimizer discipline

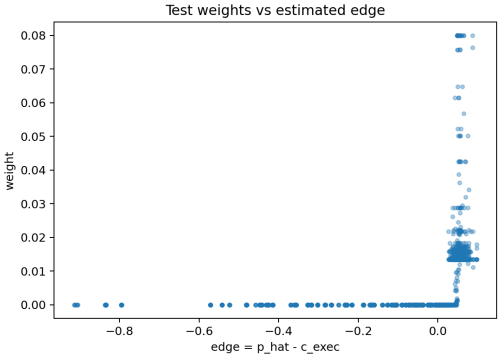
- The turnover penalty succeeds in keeping average daily reopt turnover low.
- Even so, there are bursts where the optimizer makes large portfolio changes.
- Those spikes should be treated as model-risk events: they can reflect real signal shifts, but also unstable edges or changing liquidity.

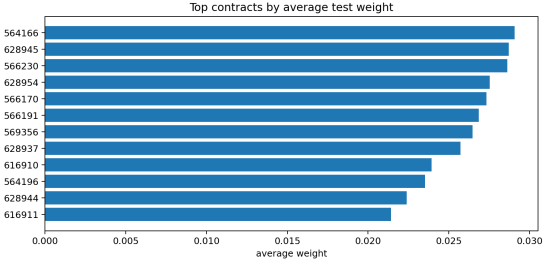


Operational lesson

Do not evaluate the model only on average objective. The turnover path is part of the strategy.

How the optimizer allocates capital





Interpretation

Weights rise with estimated edge, but not one-for-one. Contract caps, event caps, the variance penalty, and the turnover term all compress extreme allocations.

Bottom line

- The 60-day optimization formulation is coherent and implementable.
- The daily re-optimization run is the right evaluation lens for operations.
- The most important result is not just edge estimation - it is the mismatch between positive optimization objective and fragile realized pnl.

Recommendation

Use this repo as the baseline research deck for the 60-day strategy, but treat deployment as conditional on stronger realized pnl stability under stricter execution assumptions.

Methodology in one line

A compact workflow slide that makes the modeling pipeline easy to follow.

1. Panel data

275 markets across 354 dates; one row per market-day.

2. Features

Lagged returns, momentum, rolling vol, liquidity transforms, and cross-sectional ranks.

3. Validation

Train 60%, validation 20%, test 20%, in strict chronological order.

4. Scoring

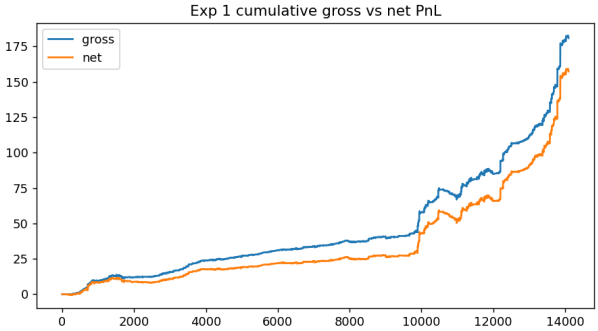
Rank IC, directional accuracy, net PnL, and experiment stress tests.

Five-experiment summary

A quick orientation slide before the deep dive.

Exp 1	Net Sharpe 0.770	Signal survives costs, but absolute fit is weak.
Exp 2	Mean bucket Rank IC 0.091	Forecastability changes across liquidity regimes.
Exp 3	Delta R^2 +0.0080	Cross-sectional features improve the baseline.
Exp 4	Coverage 0.947	Uncertainty haircuts improve reliability.
Exp 5	Val/Test corr -0.609	Validation design is the biggest fragility.

Experiment 1: persistent predictability net of costs



Out-of-sample test metrics

- Directional accuracy: 0.536
- Rank IC: 0.077
- Net Sharpe: 0.770
- Turnover: 0.262

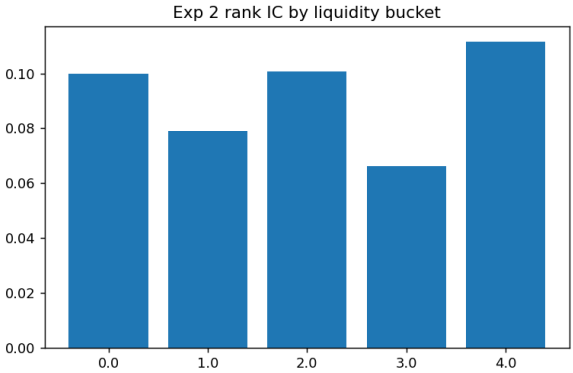
Interpretation

Absolute fit is weak, but ranking and directional signal remain useful after transaction costs.

Decile spread signal

- Bottom decile mean return: -1.71%
- Top decile mean return: 6.85%

Experiment 2: liquidity changes the signal

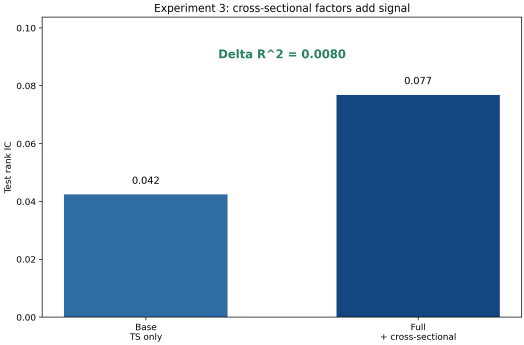


- Mean bucket Rank IC: 0.091
- Bucket-level Rank IC ranges from roughly 0.066 to 0.112.
- Forecastability is not constant across tradability regimes.

Implication

A pooled model hides useful structure. Position sizing and portfolio construction should be liquidity-aware.

Experiments 3 and 4: factor value-add and unresolved risk



- Cross-sectional features raise test Rank IC from 0.042 to 0.077.
- Incremental $\Delta R^2 = 0.0080$.

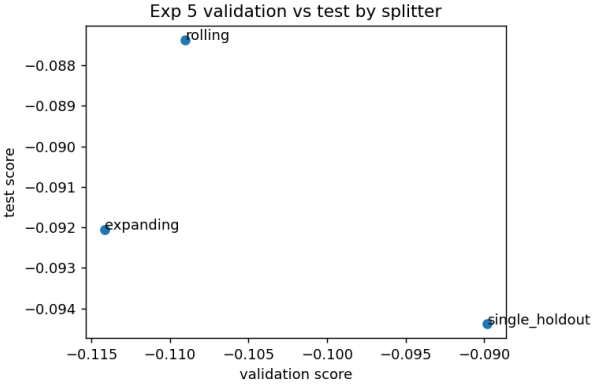
Experiment 4: unresolved-risk reliability

The uncertainty-adjusted setup improves calibration and error behavior under sparse terminal labels.

0.258	0.947	0.072
Brier score	1-sigma coverage	Adj. MAE
lower is better	near 1 = conservative	lower is better

- Brier score: 0.258
- 1-sigma coverage: 0.947
- Adjusted MAE: 0.072

Experiment 5: validation design is the main fragility



- Validation-to-test correlation is **-0.609**.
- The best validation score does *not* reliably imply the best test score.
- Rolling had the strongest test score in the table, beating single-holdout.

Takeaway

Do not trust a single holdout split for promotion decisions. Use expanding and rolling stress tests as default governance.

Main takeaways and next steps

- The updated panel pipeline produces **evidence of tradable directional edge** after costs, even though absolute fit remains weak.
- Liquidity, cross-sectional context, and unresolved-risk handling all add useful structure.
- The largest remaining risk is **validation instability**, not feature scarcity.

Next steps

Add walk-forward confidence intervals, decompose signal alpha versus cost drag, and promote only models that pass both performance and stability gates.